VIRTUAL MEMORY SYSTEMS

Virtual Memory Systems

Topics:

- Motivation for virtual memory
- Virtual vs physical addressing
- Paging
- Address translation
- Page tables
- TLB (Translation Lookaside Buffer)
- Page faults
- Replacement policies
- Thrashing
- NUMA and virtualization

Fundamental Problems

Modern systems execute:

- Multiple processes simultaneously
- Applications requiring large memory spaces
- Programs that must be isolated from each other

However

- Physical memory is limited
- Memory is fragmented
- Processes cannot directly access arbitrary physical addresses

Goals of Virtual Memory

Virtual memory provides

- Abstraction
- Protection
- Relocation
- Controlled sharing
- Larger logical address spaces

Key Idea of Virtual Memory

The main abstraction is that each process sees:

- Its own contiguous address space
- Independent from physical memory organization

The OS and HW cooperate to translate

Virtual Address to Physical Address

The program works with virtual addresses
The DRAM works with physical addresses
Translation is mandatory

Virtual vs Physical Address

Virtual Address (VA)	**Physical Address (PA)**
Generated by CPU	Used by memory hardware
Process-specific	Global system-wide
Abstraction	Actual DRAM location

Example: two processes may use the same virtual address:

Process A: VA 0x1000

Process B: VA 0x1000

but map to different physical frames.

Process Isolation

Without virtual memory:

- One process could overwrite another process memory
- Security would collapse
- Multitasking would be unsafe

With virtual memory:

- Each process has isolated mappings
- Illegal accesses generate exceptions

Page table entries contain permission bits:

- Read
- Write
- Execute

Paging

Concept: memory is divided into fixed-size blocks

Virtual pages are blocks of virtual address space

Physical frames are blocks of physical memory

Typical page sizes:

- 4 KB
- 2 MB (huge page)
- 1 GB (very large page)

Why Fixed-Size Pages?

Compared to variable-size allocation:

Advantages

- Simpler allocation
- Easier replacement
- Simpler hardware
- No external fragmentation

Disadvantages

- Internal fragmentation

Internal Fragmentation

Example: suppose

- page size = 4 KB
- Application requests 6 KB

Allocation requires 2 pages; thus, the total allocated memory is 8 KB

Unused memory:

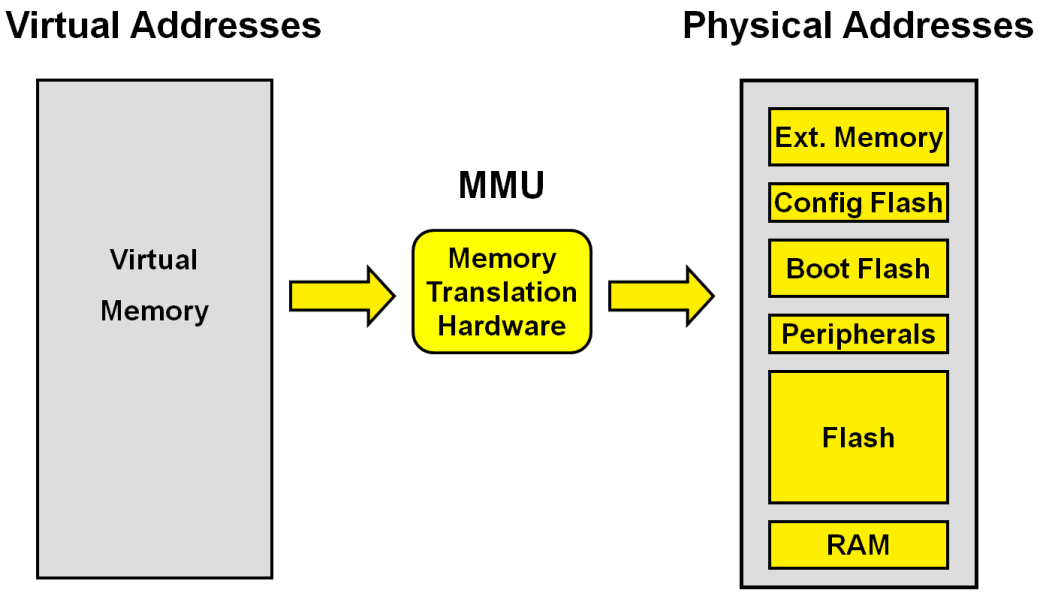
8KB-6KB=2KB

This unused space is called internal fragmentation

Address Translation

Translation mechanism:

- CPU generates a virtual address
- The MMU (Memory Management Unit):
  - Translates the address
  - Checks permissions
  - Produces the physical address



Structure of a Virtual Address

A virtual address is divided into:

- Virtual Page Number (VPN)
- Offset (byte inside page)

The offset does not change during translation, only the page number changes

Example: consider a 32-bit virtual address and a page size of 4KB

Since 4KB=2^{12} the offset is 12 bits. The remaining bits are 32-12=20

The VPN is 20 bits and the offset is 12 bits

Translation Procedure

The translation procedure

1. Extract offset
2. Extract VPN
3. Use VPN to index page table
4. Obtain PFN
5. Combine PFN and offset

Page Tables

The main idea is that page tables store mappings:

VPN → PFN

Each entry is called Page Table Entry (PTE)

PTE contains:

- Physical frame number
- Valid bit
- Dirty bit
- Access permission
- Accessed/reference bit

Single-Level Page Table

Simplest organization
One large table indexed directly by the VPN

Problem: large virtual address spaces require enormous tables

Example: consider 32-bit VA and 4KB pages

The number of pages is $2^{32}/2^{21}=2^{20}$

Page Table Size Example

Suppose:

- 1 million entries
- Each PTE is 4 bytes

The total size is $2^{20} \times 4B = 4$ MB per process

This is a problem in modern systems since they may execute

- Hundreds of processes
- Sparse address spaces

Huge memory overhead

Multi-Level Page Tables

Solution

Use hierarchical page tables

Only allocate portions actually used

Typical structure:

- Directory
- Table
- Offset

Modern x86-64 systems use 4-level paging (sometimes 5-level paging)

Multi-Level Translation

Translation Procedure:

1. Use top-level index
2. Find next-level table
3. Repeat until leaf entry
4. Obtain PFN

Benefit

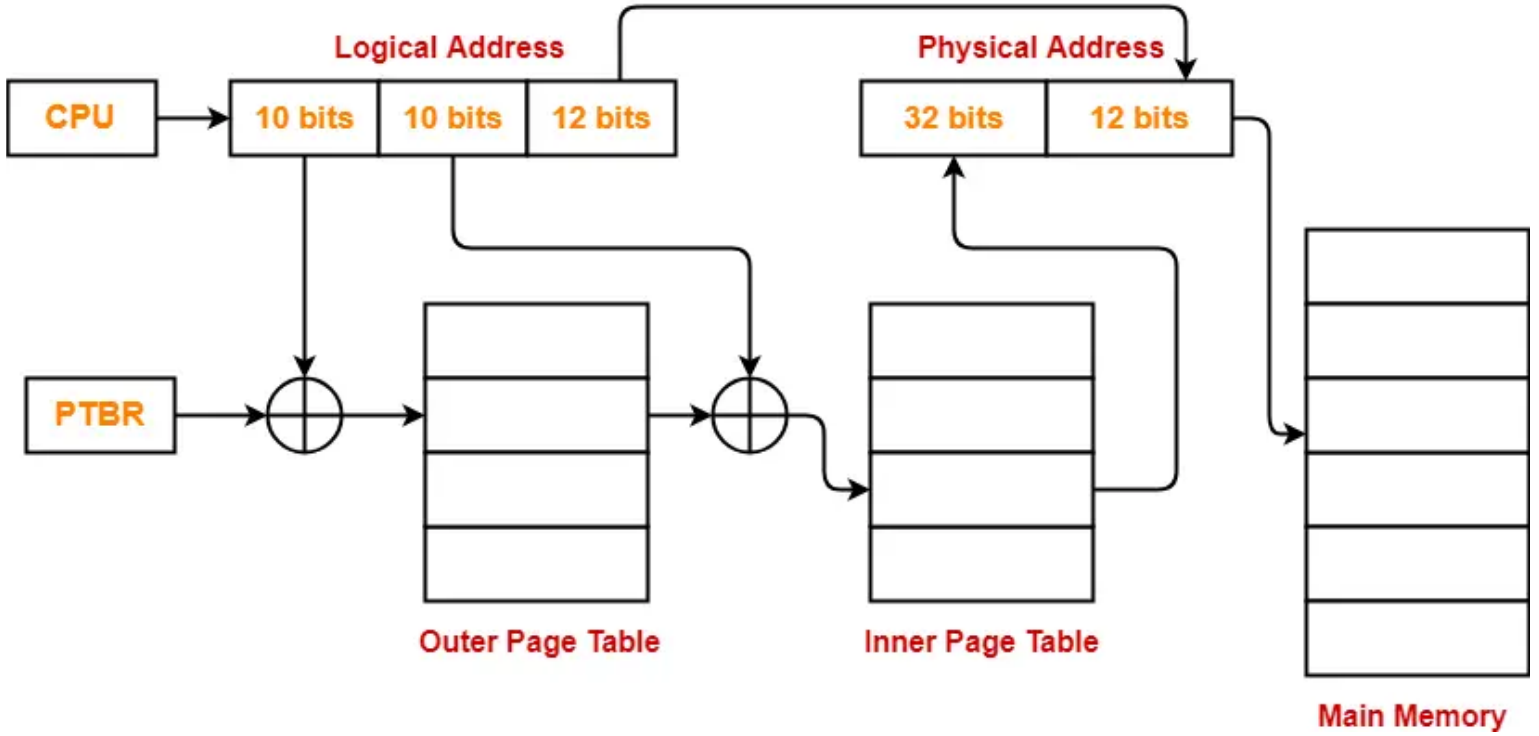
Sparse allocation

Only required page tables are allocated

Visualization of Multi-Level Paging

Key idea:

- Virtual address bits are used progressively
- Each level indexes another table
- This creates a hierarchical tree structure



Why Translation is Expensive

Without optimization:

Every memory access requires

1. Page table access
2. Actual memory access

Potentially:

$$2 \times \text{memory latency}$$

Or worse in multi-level paging

Translation Lookaside Buffer (TLB)

TLB definition: a small associative cache storing recent translations

It stores VPN → PFN

The goal is to avoid repeated page table walks

TLB hit: translation found immediately, fast access

TLB Miss: hardware or OS perform page table walk, much slower

Effective Memory Access Time

EMAT is defined as

$$EMAT = TLB_{hit\ time} + MR_{TLB} \times Penalty$$

Where MR is the miss rate and $Penalty$ is the page walk cost

Example: $TLB_{hit\ time}$ = 1 cycle, MR_{TLB} = 1% and $Penalty$ = 100 cycles

$$EMAT = 1 + 0.01 \times 100 = 2$$

TLB Reach

Definition: Amount of memory addressable by TLB whitout misses

$$TLB\ Reach = Entries\ \times Page\ Size$$

Example: with 4 KB and 512-entry TLB

$$TLB\ Reach = 521\ \times 4KB = 2MB$$

Huge Pages

Motivation: increase TLB reach

Benefits:

- Fewer TLB misses
- Smaller page tables

Costs:

- Internal fragmentation
- Allocation complexity

TLB and Multicore Systems

Problem: if page mappings change the TLB entries in multiple cores may become invalid

Solution: TLB shootdown

- OS sends invalidation requests to cores

It is expensive because it requires:

- Inter-processor interrupts
- Synchronization
- Invalidation coordination

Can become a scalability bottleneck

Page Faults

Definition: a page fault occurs when the requested page is not currently mapped in physical memory

The CPU traps into the operating system

OS actions:

1. Identify missing page
2. Locate page on disk
3. Allocate frame
4. Load page
5. Update page table
6. Resume execution

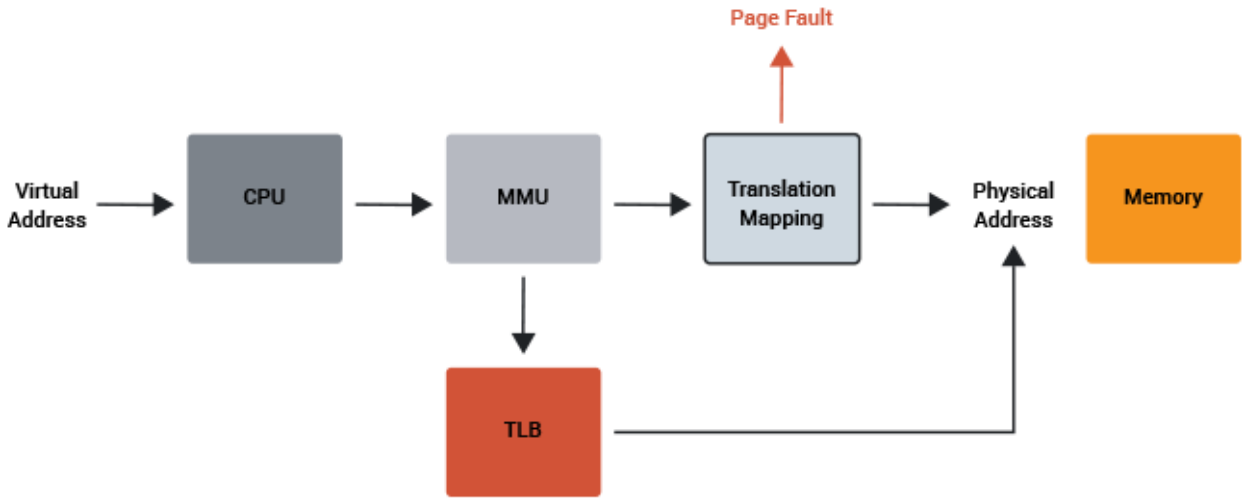
Major vs Minority

Minor fault

- Page is already available in RAM
- No disk access

Major fault

- Disk access required
- Very expensive



Demand Paging

Main idea: page are loaded only when actually accessed

Advantages:

- Lower memory usage
- Faster program startup

Drawback:

- Initial page faults

Page Replacement

Problem: Physical memory is finite and when memory is full the OS must evict pages

Common replacement policies:

- FIFO
- LRU
- Clock
- Random

Goal: minimize future page faults

Page Replacement

FIFO: evict oldest page

Advantages

- Simple
- Low overhead

Problem

- May evict frequently used pages

Last Recently Used (LRU): evict page not used recently. It is based on the temporal locality principle (recently used pages likely reused soon)

Problem: Tracking exact access order is expensive (real systems often use approximations, reference bits, aging techniques)

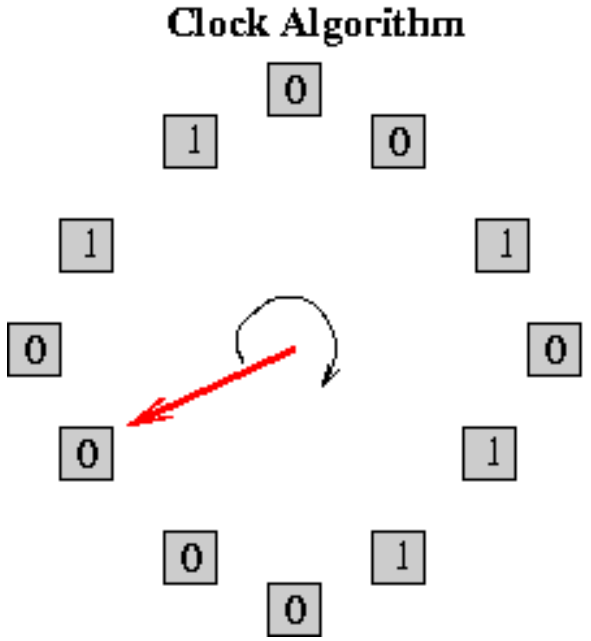
Page Replacement

Clock policy: pages arranged circularly. Reach page has a reference bit.

Algorithm:

- If bit = 0 → evict
- If bit = 1 → clear bit and continue

Cheap approximation of LRU



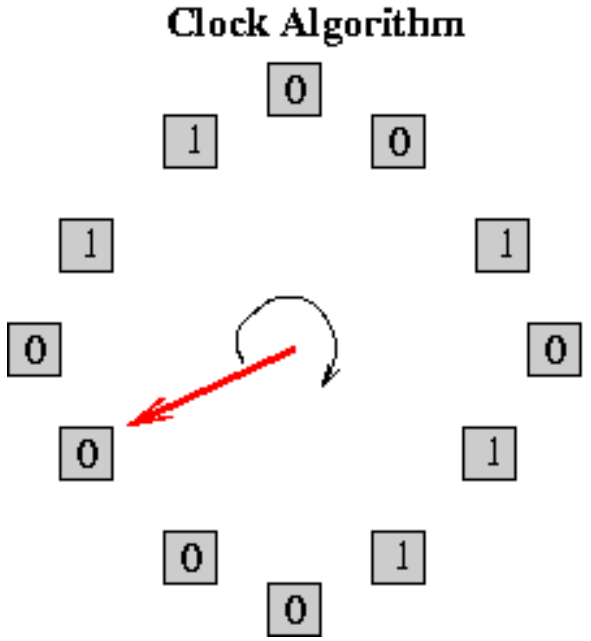
Page Replacement

Clock policy: pages arranged circularly. Reach page has a reference bit.

Algorithm:

- If bit = 0 → evict
- If bit = 1 → clear bit and continue

Cheap approximation of LRU



Thrashing

Definition: system spends more time paging than computing

Symptoms:

- Massive page faults
- Low CPU utilization
- High disk activity

Main cause:

- Working set exceeds physical memory
- System continuously swaps pages

Working Set Model

The working set is the set of actively used pages

If working set fits in memory → low page fault rate
Otherwise → thrashing likely

Virtual memory interacts with:

- L1/L2/L3 caches
- TLB
- Coherence
- Memory controllers

NUMA systems

Non-Uniform Memory Access

Modern systems have:

- Multiple sockets
- Multiple memory controllers
- Local and remote memory

Latency depends on memory location

Remote accesses feature higher latency and lower bandwidth

OS and applications must preserve locality

Virtualization

Virtual machines introduce:

- Guest virtual addresses
- Guest physical addresses
- Host physical addresses

Additional translation layer required

Two level translation

- Guest OS performs one translation
- Hypervisor performs another
- Potentially large overhead

Modern CPUs provide hardware acceleration

Modern Perspective

Virtual memory is not isolated

It interacts with:

- OS
- Compilers
- Caches
- TLBs
- Multicore systems
- Virtualization

Summary

Virtual memory is

- A protection mechanism
- An abstraction mechanism
- A performance problem
- A HW/SW co-design challenge

It is one of the most important abstractions in modern computing systems.

Next episode: DRAM architecture and memory controllers